

## Segunda práctica Más Lomo

Primer miembro: Miguel Barranquero Díez (i0917650)

Segundo miembro: Alejandro González Rengel (i0970320)

09 de Mayo de 2023

//SEGUNDA PRÁCTICA EVALUABLE · SSOOII · MAS LOMO · Miguel Barranquero Diez ·  
Alejandro González Rengel

```
01 #include <iostream>
02 #include <time.h>
03 #include "windows.h"
04 #include "lomo2.h"

05 #define LOGIN1 "i0970320"
06 #define LOGIN2 "i0917650"

07 #define NUM_TREN_MAX 100
08 #define NUM_TREN_MIN 0

09 #define ROWS 17
10 #define COLUMNS 75

11 #pragma region DLL

12 HINSTANCE libreria = LoadLibrary(TEXT("../libs/lomo2.dll"));

13 int(*GenerarMapa)(char const*, char const*);
14 int(*Iniciar)(int, int, char const*, char const*);

15 int(*PetAvance)(int, int*, int*);
16 int(*Avance)(int, int*, int*);

17 char* (*GetColor)(int);
18 VOID(*Espera)(int, int);

19 void(*Error)(const char*);
20 int(*TrenNuevo)(VOID);
21 int(*Fin)(VOID);

22 #pragma endregion

23 CRITICAL_SECTION scInicio;
24 CRITICAL_SECTION scMapa;
25 HANDLE empezar;
26 int salir_bucle = 1;
27 int bloqueado = 0;

28 #pragma region ENCABEZADOS FUNCIONES
29 int cargarDLL();

30 DWORD WINAPI funcionTrenes(LPVOID a);
31 BOOL WINAPI CtrlHandler(DWORD serial);

32 #pragma endregion

33 int main(int argc, const char* argv[])
34 {
35     if (libreria == NULL)
36     {
37         printf("No encuentro la DLL lomo2\r\n");
38         fflush(stdout);
39         return -1;
40     }
```

```

41     if (cargarDLL() == -1)
42     {
43         Error("ERROR al cargar lomo2.dll");
44         FreeLibrary(libreria);
45         return 0;
46     }

47     if (argc == 1 || argc > 4)
48     {
49         printf("Error, numero de argumentos incorrectos\n\r");
50         fflush(stdout);
51         FreeLibrary(libreria);
52         return -1;
53     }

54     if (strcmp(argv[1], "--mapa") == 0)
55     {
56         if (GenerarMapa(LOGIN1, LOGIN2) == -1)
57         {
58             Error("Error al generar MAPA LOMO\r\n");
59         }
60         FreeLibrary(libreria);
61         return 0;
62     }

63     if (argc == 4)
64     {
65         MSG msj;
66         int mapa[1275];
67         HANDLE trenes[NUM_TREN_MAX];
68         DWORD trenesID[NUM_TREN_MAX];
69         int retardo = atoi(argv[1]);
70         int tamanoTren = atoi(argv[2]);
71         int nTrenes = atoi(argv[3]);

72         if (retardo < 0)
73         {
74             puts("ERROR: Valor de retardo incorrecto\r\n");
75             fflush(stdout);
76             FreeLibrary(libreria);
77             exit(1);
78         }
79         else if ((nTrenes < NUM_TREN_MIN) || (nTrenes > NUM_TREN_MAX))
80         {
81             puts("ERROR: Numero de trenes incorrecto\r\n");
82             fflush(stdout);
83             FreeLibrary(libreria);
84             exit(1);
85         }
86         else if ((tamanoTren < 3) || (tamanoTren > 19))
87         {
88             puts("ERROR: tamano de los trenes incorrecto\r\n");
89             fflush(stdout);
90             FreeLibrary(libreria);
91             exit(1);
92         }

93         InitializeCriticalSection(&scInicio);
94         InitializeCriticalSection(&scMapa);

```

```

95     empezar = CreateEvent(NULL, TRUE, FALSE, "trenesEvent");
96     if (empezar == NULL)
97     {
98         Error("Error Evento");
99         DeleteCriticalSection(&scMapa);
100        DeleteCriticalSection(&scInicio);
101        FreeLibrary(libreria);
102        exit(-1);
103    }

104    HANDLE memmoriaCompartida;
105    LPINT  puntMem;

106    if (INVALID_HANDLE_VALUE == (memmoriaCompartida =
CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0,
sizeof(mapa), "memP")))
107    {
108        Error("Error al crear segmento de memoria compartida");

109        DeleteCriticalSection(&scMapa);
110        DeleteCriticalSection(&scInicio);
111        CloseHandle(empezar);
112        FreeLibrary(libreria);
113        exit(2);
114    }

115    if (NULL == (puntMem = (LPINT)MapViewOfFile(memmoriaCompartida,
FILE_MAP_ALL_ACCESS, 0, 0, sizeof(mapa))))
116    {
117        Error("Error al mapear memoria");

118        CloseHandle(memmoriaCompartida);
119        DeleteCriticalSection(&scMapa);
120        DeleteCriticalSection(&scInicio);
121        CloseHandle(empezar);
122        FreeLibrary(libreria);
123        exit(-1);
124    }

125    for (int i = 0; i < ROWS; i++)
126    {
127        for (int j = 0; j < COLUMNS; j++)
128        {
129            mapa[i * COLUMNS + j] = 0; //inic. cada posicion a 0
130        }
131    }
132    CopyMemory(puntMem, &mapa, sizeof(mapa));

133    if (-1 == Iniciar(retardo, tamanoTren, LOGIN1, LOGIN2))
134    {
135        Error("ERROR AL INICIAR LOMO\n");

136        CloseHandle(memmoriaCompartida);
137        UnmapViewOfFile(puntMem);
138        DeleteCriticalSection(&scMapa);
139        DeleteCriticalSection(&scInicio);
140        CloseHandle(empezar);
141        FreeLibrary(libreria);
142        exit(1);
143    }

```

```

144     for (int i = 0; i < nTrenes; i++)
145     {
146         trenes[i] = CreateThread(NULL, 0, funcionTrenes, LPVOID(i), 0,
            &trenesID[i]);

147         if (trenes[i] == NULL)
148         {
149             Error("ERROR, al crear tren %d\r\n");

150             CloseHandle(memmoriaCompartida);
151             UnmapViewOfFile(puntMem);
152             DeleteCriticalSection(&scMapa);
153             DeleteCriticalSection(&scInicio);
154             CloseHandle(empezar);
155             FreeLibrary(libreria);
156         }
157     }

158     SetEvent(empezar);
159     SetConsoleCtrlHandler(CtrlHandler, TRUE);

160     clock_t startTime = clock();
161     clock_t endTime = startTime + 60 * CLOCKS_PER_SEC;

162     while (clock() < endTime)
163     {
164         if (salir_bucle == 0)
165         {
166             break;
167         }
168     }

169     for (int i = 0; i < nTrenes; i++)
170     {
171         PostThreadMessage(trenesID[i], 110, WPARAM(0), LPARAM(0));
172     }

173     WaitForMultipleObjects(nTrenes, trenes, true, INFINITE);
174     if (Fin() == 0)
175     {
176         Error("EJECUCION CORRECTA");
177         system("cls");
178     }

179     for (int j = 0; j < nTrenes; j++)
180     {
181         CloseHandle(trenes[j]);
182     }

183     CloseHandle(memmoriaCompartida);
184     CloseHandle(empezar);
185     UnmapViewOfFile(puntMem);

186     DeleteCriticalSection(&scMapa);
187     DeleteCriticalSection(&scInicio);

188     FreeLibrary(libreria);
189 }
190 }
191 }
192 return 0;
193 }

```

```

194 DWORD WINAPI funcionTrenes(LPVOID a)
195 {

196     MSG msj;
197     HANDLE memoria = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE, "memP");

198     if (memoria == INVALID_HANDLE_VALUE)
199     {
200         Error("ERROR al abrir memoria compartida");
201         fflush(stdout);
202         return -1;
203     }

204     LPINT m = (LPINT)MapViewOfFile(memoria, FILE_MAP_ALL_ACCESS, 0, 0,
        sizeof(int) * 1275);

205     if (m == NULL)
206     {
207         CloseHandle(memoria);
208         ExitThread(-1);
209     }

210     DWORD tiempoInicio;
211     DWORD tiempoFinal;
212     DWORD tiempoEspera;
213     HANDLE con = GetStdHandle(STD_OUTPUT_HANDLE);

214     int idTren;

215     int xcab = 0;
216     int xcola = 0;
217     int ycab = 0;
218     int ycola = 0;

219     int libre = 0;
220     int creado = 0;

221     int ycabezaAntes = 0;

222     WaitForSingleObject(empezar, INFINITE);
223     EnterCriticalSection(&scInicio);

224     idTren = TrenNuevo();

225     if (idTren == -1)
226     {
227         Error("Error al asignar identificador al tren");
228         CloseHandle(memoria);
229         UnmapViewOfFile(m);
230         CloseHandle(con);
231         ExitThread(-1);
232     }

233     const char* color = GetColor(idTren);

```

```

234 while (true)
235 {
236     tiempoInicio = GetTickCount();

237     libre = 0;
238     ycabezaAntes = ycab;

239     PetAvance(idTren, &xcab, &ycab);

240 do
241 {
242     if (PeekMessage(&msj, NULL, 110, 110, PM_NOREMOVE))
243     {
244         CloseHandle(memoria);
245         UnmapViewOfFile(m);
246         ExitThread(1);
247     }

248     EnterCriticalSection(&scMapa);

249     tiempoFinal = GetTickCount();
250     tiempoEspera = tiempoFinal - tiempoInicio;

251     if (tiempoEspera > 900 && bloqueado == 0)
252     {
253         bloqueado = 1;
254         printf("\n\n TREN BLOQUEADO, ID: %d ---- ", idTren);

255         if (strcmp(color, "verde") == 0)
256         {
257             SetConsoleTextAttribute(con, FOREGROUND_GREEN);
258         }
259         else if (strcmp(color, "blanco") == 0)
260         {
261             SetConsoleTextAttribute(con, FOREGROUND_RED |
262                                     FOREGROUND_GREEN | FOREGROUND_BLUE);
263         }
264         else if (strcmp(color, "morado") == 0)
265         {
266             SetConsoleTextAttribute(con, FOREGROUND_RED |
267                                     FOREGROUND_BLUE);
268         }
269         else if (strcmp(color, "cielo") == 0)
270         {
271             SetConsoleTextAttribute(con, FOREGROUND_BLUE);
272         }
273         else if (strcmp(color, "amarillo") == 0)
274         {
275             SetConsoleTextAttribute(con, FOREGROUND_RED |
276                                     FOREGROUND_GREEN);
277         }
278         else if (strcmp(color, "rojo") == 0)
279         {
280             SetConsoleTextAttribute(con, FOREGROUND_RED);
281         }
282         printf("COLOR: %s", color);
283         SetConsoleTextAttribute(con, FOREGROUND_RED |
284                                 FOREGROUND_GREEN | FOREGROUND_BLUE);

285         salir_bucle = 0;
286     }
287 }

```

```

283         m = m + (ycab * COLUMNS + xcab);

284         if (*m == 0)
285         {
286             *m = 1;
287             m = m - (ycab * COLUMNS + xcab);
288             LeaveCriticalSection(&scMapa);
289             libre = 1;
290         }
291         else
292         {
293             m = m - (ycab * COLUMNS + xcab);
294             LeaveCriticalSection(&scMapa);
295         }
296     } while (libre == 0);

297     Avance(idTren, &xcola, &ycola);

298     if (xcola != -1 && creado == 0)
299     {
300         LeaveCriticalSection(&scInicio);
301         creado = 1;
302     }
303     else if (xcola != -1)
304     {
305         EnterCriticalSection(&scMapa);
306         m = m + (ycola * COLUMNS + xcola);
307         *m = 0;
308         m = m - (ycola * COLUMNS + xcola);
309         LeaveCriticalSection(&scMapa);
310     }
311     Espera(ycabezaAntes, ycab);
312 }
313 }

314 BOOL WINAPI CtrlHandler(DWORD senial)
315 {
316     if (senial == CTRL_C_EVENT)
317     {
318         salir_bucle = 0;
319         return true;
320     }

321     return false;
322 }

323 int cargarDLL()
324 {
325     GenerarMapa = (int (*)(char const*, char const*))GetProcAddress(libreria,
        "LOMO_generar_mapa");
326     if (GenerarMapa == NULL)
327     {
328         printf("Error al cargar GenerarMapa() de lomo2.dll\r\n");
329         fflush(stdout);
330         return -1;
331     }

332     Iniciar = (int (*)(int, int, char const*, char
        const*))GetProcAddress(libreria, "LOMO_inicio");
333     if (Iniciar == NULL)
334     {

```

```

335         printf("Error Bindeo Iniciar\r\n");
336         fflush(stdout);
337         return -1;
338     }

339     TrenNuevo = (int(*) (VOID))GetProcAddress(libreria, "LOMO_trenNuevo");
340     if (Iniciar == NULL)
341     {
342         printf("Error Bindeo Iniciar\r\n");
343         fflush(stdout);
344         return -1;
345     }

346     PetAvance = (int(*) (int, int*, int*))GetProcAddress(libreria,
"LOMO_peticiOnAvance");
347     if (PetAvance == NULL)
348     {
349         printf("Error al cargar LOMO_peticiOnAvance() de lomo2.dll\r\n");
350         fflush(stdout);
351         return -1;
352     }

353     Avance = (int(*) (int, int*, int*))GetProcAddress(libreria, "LOMO_avance");
354     if (Avance == NULL)
355     {
356         printf("Error al cargar LOMO_avance() de lomo2.dll\r\n");
357         fflush(stdout);
358         return -1;
359     }

360     GetColor = (char* (*) (int))GetProcAddress(libreria, "LOMO_getColor");
361     if (GetColor == NULL)
362     {
363         printf("Error al cargar LOMO_getColor() de lomo2.dll\r\n");
364         fflush(stdout);
365         return -2;
366     }

367     Espera = (VOID(*) (int, int))GetProcAddress(libreria, "LOMO_espera");
368     if (Espera == NULL)
369     {
370         printf("Error al cargar LOMO_espera() de lomo2.dll\r\n");
371         fflush(stdout);
372         return -2;
373     }

374     Error = (VOID(*) (const char*))GetProcAddress(libreria, "pon_error");
375     if (Error == NULL)
376     {
377         printf("Error al cargar LOMO_espera() de lomo2.dll\r\n");
378         fflush(stdout);
379         return -2;
380     }

381     Fin = (int(*) (VOID))GetProcAddress(libreria, "LOMO_fin");
382     if (Fin == NULL)
383     {
384         printf("Error al cargar LOMO_fin() de lomo2.dll\r\n");
385         fflush(stdout);
386         return -2;
387     }
388     return 0;
389 } /*fin*/

```